LoadScope: Phase 1 Deep-Dive

Core Runner, DSA Histogram & Closed-Model Architecture

An exhaustive, line-by-line engineering breakdown of Phase 1 implementation.

What Was Built in Phase 1?

Phase 1 transforms LoadScope from a scaffolding project into a high-performance load-testing engine. It implements the custom fixed-memory log-bucketed histogram, closed-model concurrency loop with undici connection pooling, high-precision nanosecond timing, and full terminal reporting.

Ø=Ü¡ Phase 1 Definition of Done (Passed)
Phase 1 is complete when: (1) Custom histogram is verified against hdr-histogram-js within 3%, (2) Closed-model runner executes against mock server, (3) Terminal displays latency percentiles and RPS, and (4) All automated unit/integration tests pass.

Files Built in Phase 1

Every component built in Phase 1 fulfills a specific role in the core load-testing engine:

- **src/stats/histogram.ts:** Fixed-memory log-bucketed histogram using 1,200 exponential buckets (4.8 KB Uint32Array).
- **src/config/schema.ts:** Zod schema validating URLs, methods, headers, and duration strings (e.g. '30s', '1m').
- **src/core/runner.ts:** Closed-model load engine using undici.Pool, connection keep-alive, and monotonic timing.
- **src/report/terminal.ts:** Terminal UI formatter displaying latency percentiles (p50, p95, p99), RPS, and status codes.
- **src/cli.ts:** Upgraded CLI entrypoint accepting all user flags (-u, -c, -d, -m, -H, -b).
- **test/histogram.test.ts:** Unit tests validating custom histogram percentiles against hdr-histogram-js.
- **test/runner.test.ts:** Integration tests against the mock server (root 200, delay 30ms, error 500).

1. The Fixed-Memory Histogram (src/stats/histogram.ts)

The fixed-memory histogram is the core data structure of LoadScope. Naive arrays cause O(N) memory growth and GC pauses. This custom histogram keeps memory flat at 4.8 KB forever.

```
export const GROWTH = 1.02;
export const BUCKETS = 1200; // 1.02^1200 us "H 21,114 seconds (~5.8 hours)

export class Histogram {
  readonly counts = new Uint32Array(BUCKETS); // 1,200 * 4 bytes = 4.8 KB
  total = 0;
  private minVal = Infinity;
  private maxVal = 0;
  private sumUs = 0;

  private idx(us: number): number {
    if (us <= 1) return 0;
    const bucket = Math.floor(Math.log(us) / Math.log(GROWTH));
    return Math.min(BUCKETS - 1, Math.max(0, bucket));
  }

  private valFromIdx(i: number): number {
    return Math.pow(GROWTH, i + 1);
  }

  record(us: number): void {
    const val = Math.max(0, us);
    const i = this.idx(val);
    this.counts[i]++;
    this.total++;
    this.sumUs += val;
    if (val < this.minVal) this.minVal = val;
    if (val > this.maxVal) this.maxVal = val;
  }
}
```

- **const GROWTH = 1.02:** Geometric growth ratio between adjacent buckets. Each bucket boundary is 2% higher than the previous, bounding relative error to ~2%.
- **const BUCKETS = 1200:** Total bucket count. Because 1.02^1200 "H 2.11 × 10^10 microseconds (~5.8 hours), it covers all conceivable web request latencies without overflowing.
- **counts = new Uint32Array(1200):** Fixed typed array. Storing 1,200 unsigned 32-bit integers takes exactly 4,800 bytes (4.8 KB). No V8 heap resizing occurs regardless of how many millions of requests are run.
- **Math.log(us) / Math.log(GROWTH):** Change of base formula: log_1.02(us). Maps any latency in microseconds to an integer bucket index in O(1) time.
- **Math.pow(GROWTH, i + 1):** Inverse mapping: Reconstructs microsecond latency from a bucket index during percentile extraction.
- **this.counts[i]++:** O(1) recording: Increments the frequency count for the corresponding bucket.

2. Histogram Percentile Extraction & Merging

Percentiles (p50, p95, p99) are extracted by walking the 1,200 buckets cumulatively until the target count is satisfied.

```
percentile(p: number): number {
  if (this.total === 0) return 0;
  if (p <= 0) return this.min();
  if (p >= 100) return this.max();

  const target = Math.ceil((p / 100) * this.total);
  let seen = 0;

  for (let i = 0; i < BUCKETS; i++) {
    seen += this.counts[i];
    if (seen >= target) {
      return this.valFromIdx(i);
    }
  }
  return this.max();
}

merge(other: Histogram): void {
  for (let i = 0; i < BUCKETS; i++) {
    this.counts[i] += other.counts[i];
  }
  this.total += other.total;
  this.sumUs += other.sumUs;
  if (other.minVal < this.minVal) this.minVal = other.minVal;
  if (other.maxVal > this.maxVal) this.maxVal = other.maxVal;
}
```

- **target = Math.ceil((p / 100) * this.total):** Computes the ordinal rank for the percentile. For example, for p95 with 10,000 samples, target = 9,500.
- **for (let i = 0; i < BUCKETS; i++):** Sequential scan over 1,200 integers. In modern CPUs, scanning 1,200 memory-contiguous integers takes less than 1 microsecond.
- **if (seen >= target):** The moment cumulative sum reaches target, the current bucket index represents that percentile latency.
- **merge(other: Histogram):** O(B) vector addition: Adds bucket counts element-by-element. This allows multiple worker threads to merge stats with zero locking!

Ø=Ü¡ Why Percentiles in Microseconds?

We record and calculate in microseconds (us) to preserve sub-millisecond precision. We provide .percentileMs(p) to divide by 1,000 for user-friendly millisecond output.

3. Option Validation & Durations (src/config/schema.ts)

Robust CLI tools must handle diverse user inputs gracefully: '30s', '1m', '500ms', multiple headers, and invalid URLs.

```
export function parseDurationToSeconds(val: string | number): number {
  if (typeof val === "number") return val;
  const trimmed = val.trim().toLowerCase();
  const match = trimmed.match(/^(\d+(?:\.\d+)?)\s*(ms|s|m|h)?$/);
  if (!match) throw new Error(`Invalid duration format: "${val}"`);
  const num = parseFloat(match[1]);
  const unit = match[2] || "s";
  switch (unit) {
    case "ms": return num / 1000;
    case "s":  return num;
    case "m":  return num * 60;
    case "h":  return num * 3600;
    default:   return num;
  }
}

export const RunOptionsSchema = z.object({
  url: z.string().url("A valid URL starting with http:// or https:// is required"),
  duration: z.union([z.string(), z.number()]).default("10s").transform(parseDurationToSeconds),
  concurrency: z.coerce.number().int().min(1).default(10),
  rate: z.coerce.number().positive().optional(),
  method: z.enum(["GET", "POST", "PUT", "DELETE", "PATCH", "HEAD", "OPTIONS"]).default("GET"),
  headers: z.record(z.string()).default({}),
  body: z.string().optional(),
});
```

- **parseDurationToSeconds:** Regex parser matching numbers and optional unit suffixes (ms, s, m, h). Converts everything to numeric seconds.
- **z.string().url():** Guarantees the target URL includes a valid protocol (http:// or https://) before attempting any socket connections.
- **.transform(parseDurationToSeconds):** Zod pipeline transforms raw user strings like '30s' into numeric 30 automatically during schema validation.
- **z.coerce.number():** Coerces string CLI arguments (e.g. from -c '50') into valid integers safely.

4. Core Closed-Model Runner (src/core/runner.ts)

The runner orchestrates undici connection pooling, concurrent virtual users, nanosecond monotonic timing, and metric aggregation.

```
export async function executeClosedModel(options: ValidatedRunOptions): Promise<RunSummary> {
  const urlObj = new URL(options.url);
  const pool = new Pool(urlObj.origin, {
    connections: options.concurrency,
    pipelining: 1,
    keepAliveTimeout: 30_000,
    keepAliveMaxTimeout: 60_000,
  });

  const histogram = new Histogram();
  const statusCodes: Record<number, number> = {};
  let errorCount = 0;
  const series: LatencyPoint[] = [];

  const durationNs = BigInt(Math.floor(options.duration * 1_000_000_000));
  const startNs = process.hrtime.bigint();
  const endNs = startNs + durationNs;
  let stopped = false;

  async function workerLoop(): Promise<void> {
    while (!stopped) {
      if (process.hrtime.bigint() >= endNs) { stopped = true; break; }
      const reqStartNs = process.hrtime.bigint();
      try {
        const response = await pool.request({
          path: urlObj.pathname + urlObj.search,
          method: options.method as any,
          headers: options.headers,
          body: options.body,
        });
        await response.body.dump(); // Consumes socket immediately
        const elapsedUs = Number(process.hrtime.bigint() - reqStartNs) / 1000;
        histogram.record(elapsedUs);
        const code = response.statusCode;
        statusCodes[code] = (statusCodes[code] || 0) + 1;
      } catch {
        errorCount++;
      }
    }
  }

  const workers = Array.from({ length: options.concurrency }, () => workerLoop());
  await Promise.all(workers);
  await pool.close();
  ...
}
```

5. Inside the Closed-Model Runner

- **const pool = new Pool(urlObj.origin, ...):** Creates an undici connection pool. undici is Node.js's fastest HTTP/1.1 client with native connection pooling and zero extra stream overhead.
- **connections: options.concurrency:** Limits open TCP sockets to match concurrency. Keeps connections open via HTTP Keep-Alive (30s timeout), avoiding repetitive TCP 3-way handshakes.
- **process.hrtime.bigint():** Returns monotonic nanoseconds (10^-9 s). Unlike Date.now(), hrtime cannot go backwards due to NTP synchronization or system clock changes.
- **async function workerLoop():** Simulates one virtual user. Sends a request, waits for response, records latency into the histogram, and immediately loops to send the next request.
- **await response.body.dump():** Crucial performance line. Discards the HTTP response body immediately without buffering it in memory, releasing the TCP socket back to the pool instantly.
- **histogram.record(elapsedUs):** Calculates elapsed microseconds from nanosecond delta: (nowNs - startNs) / 1000, and records it in O(1).
- **await Promise.all(workers):** Runs all N concurrent worker loops in parallel and waits for all of them to cleanly finish after the duration expires.
- **await pool.close():** Gracefully closes all open HTTP keep-alive sockets and TCP connections.

Live Benchmark Results on Phase 1 Completion

When we verified Phase 1 against our mock server, LoadScope delivered extraordinary throughput:

```
Duration:    2.0s    Total Requests: 28,405
Throughput:  14,161.2 req/s    Error Rate:     0.00%
Latency Distribution:
  p50 (median):  1.18 ms
  p95:           2.49 ms
  p99:           5.62 ms
HTTP Status Codes:
  200: 28,405 requests
```

6. Automated Testing Suite & Validation

Validating Custom Histogram against hdr-histogram-js

In test/histogram.test.ts, we generate 10,000 synthetic latency samples with a realistic bimodal distribution (90% fast 5-30ms requests, 10% slow tail 100-500ms requests) and compare our percentiles against the industry-standard HDR Histogram library:

```
const testPercentiles = [50, 75, 90, 95, 99];
for (const p of testPercentiles) {
  const customVal = custom.percentile(p);
  const refVal = reference.getValueAtPercentile(p);
  const relativeError = Math.abs(customVal - refVal) / refVal;
  expect(relativeError).toBeLessThan(0.03); // Strictly bounded within 3%
}
```

Summary of Phase 1 Achievements

- **Fixed Memory Footprint:** The histogram occupies strictly 4.8 KB forever. Zero GC impact.
- **High Throughput:** Exceeds 14,000 req/s on a single thread against local endpoints.
- **Accurate Tail Latency:** Verified p50, p95, and p99 against standard HDR algorithms.
- **Controllable Timing:** Verified accurate latency tracking on /delay/:ms endpoints.
- **Rock-Solid Testing:** 13/13 passing automated unit and integration tests.